



**POLYTECHNIQUE
MONTRÉAL**

**UNIVERSITÉ
D'INGÉNIERIE**

INF6103 : Cybersécurité des Infrastructures Critiques

TP 1 : Cybersécurité des infrastructures critiques infonuagiques

Introduction

De plus en plus d'entreprises remplacent des serveurs locaux pour confier la gestion de leurs données à l'infonuagique. Cette tendance s'étend aujourd'hui à de nombreuses infrastructures critiques. À titre d'exemple, on peut citer le secteur maritime qui recommande l'utilisation du PPU (Portable Pilot Unit). Un PPU permet à un pilote ayant en charge un navire, d'accéder dans le nuage, à l'ensemble des données de navigation relatives à sa mission. Le PPU représente ainsi un excellent système d'aide au pilotage, notamment dans des conditions de visibilité limitée. [Plus d'information ici](#).

Dans le secteur médical, l'utilisation de l'infonuagique se développe également. Dans les systèmes d'imagerie médicale basés sur l'infonuagique, les fonctions de stockage des dossiers médicaux et de traitement des images sont fournies par des serveurs externalisés dans le nuage. L'infonuagique apporte de nouvelles solutions aux besoins de stockage croissants de données des centres de radiologie et remplace les serveurs PACS (Picture Archiving and Communication System) locaux et jusque-là confinés dans les établissements de radiologie. [Plus d'information ici](#).

Cependant, l'utilisation de l'infonuagique dans des secteurs critiques n'est pas sans risque. Le manque ou l'absence de sécurité peut créer des brèches dont l'exploitation peut avoir des conséquences catastrophiques. À titre d'exemple, l'incident survenu le 5 février 2021 dans la centrale de traitement de l'eau de la ville de Oldsmar en Floride. Un individu non identifié a tenté de changer le dosage d'hydroxyde de sodium, ce qui aurait eu pour conséquence d'empoisonner l'eau si la malveillance n'avait pas été détectée par les personnels de la centrale. L'attaquant a profité d'une mauvaise gestion des mots de passe, de l'absence de pare-feu et d'un système d'exploitation dépassé reposant sur Windows 7. Il a ainsi pu accéder au système SCADA de la centrale en passant par le logiciel de télémaintenance TeamViewer. [Plus d'information ici](#).

Dans ce TP, vous incarnez tantôt le rôle d'un administrateur d'infrastructure infonuagique, tantôt celui d'un pentester. Hazardous Chemicals Corporation (HCC), une entreprise de fabrication de produits chimiques dangereux, a entrepris la migration de son infrastructure vers le nuage informatique. Cette infrastructure est principalement composée de systèmes de contrôle industriel qui font fonctionner les cuves de produits chimiques. Afin d'optimiser les coûts et les performances, HCC opte pour une solution reposant sur des conteneurs et hébergée chez le fournisseur de service Cloud Solutions Provider (CSP). Vous êtes sollicités pour mener à bien le déploiement de l'infrastructure de HCC sur le nuage informatique de CSP.

Le présent TP se concentre sur les aspects de sécurité liés aux applications déployées dans l'infonuagique. Plus spécifiquement, on considère des applications réalisées à l'aide de la technologie de conteneurs. Plus d'informations sur les conteneurs et leur orchestration sont disponibles en ligne sur :

- <https://www.docker.com/101-tutorial>
- <https://kubernetes.io/docs/concepts/>

Le TP se compose de 3 parties :

1. Prise en main de l'environnement conteneurisé : Docker et Minikube.
2. Sécurité de l'API de l'orchestrateur et contrôle d'accès.
3. Test de pénétration sur l'infrastructure infonuagique.

Étape Préliminaire

Cette étape doit être effectuée avant la première séance du TP. L'environnement du TP consiste en une machine virtuelle avec Minikube fonctionnant avec le driver Docker :

1. Préparation d'une machine virtuelle :
 - 3 vCPUs
 - 4 Go de mémoire vive
 - 20 Go de disque dur
 - 2 interfaces réseaux (NAT et privé hôte)
 - Système d'exploitation : [Ubuntu Server 26.04 LTS](#)
 - Installation minimale avec [OpenSSH](#)
2. Installation de l'environnement Docker : <https://docs.docker.com/engine/install/ubuntu/>
ensuite <https://docs.docker.com/engine/install/linux-postinstall/>
3. Installation de Minikube : <https://minikube.sigs.k8s.io/docs/start/>
4. Installation de Kubectl :
<https://kubernetes.io/fr/docs/tasks/tools/install-kubectl/>
5. Test de l'environnement avec `minikube start --driver=docker`.
6. Installation de Skopeo et de GnuPG : `sudo apt install skopeo gnupg`

NOTE : Pour vous faciliter la tâche, connectez-vous à la machine virtuelle en ssh après l'avoir installée.

Première séance de TP

1 Première partie [13/33]

Cette partie du TP aborde :

- 1.1 La manipulation des conteneurs Docker.
- 1.2 La validation de l'intégrité et de l'authenticité des images Docker.
- 1.3 La manipulation basique d'objets Kubernetes.

1.1 Opérations sur des images Docker [4 points]

La première étape sera de vous assurer que la plateforme de conteneurisation est opérationnelle. Pour cela, utilisez la commande `docker info`.

1.1.1 Exécutez successivement les commandes suivantes :

```
docker run --name test-busybox busybox
docker run --name test-busybox busybox
```

Expliquez le résultat de chaque commande. Distinguez clairement l'image Docker, le conteneur créé et le nom attribué au conteneur. Supprimez ensuite le conteneur sans supprimer l'image. *[0.5 point]*

Votre tâche est maintenant de déployer le serveur de surveillance vidéo des cuves de produits chimiques. Vous devez l'installer à partir du Dockerfile présent dans le dépôt Github suivant : <https://github.com/mamerzouk/hcc>.

1.1.2 Analysez le Dockerfile du serveur de vidéosurveillance. Pour chaque instruction, précisez son rôle pendant la construction de l'image. *[1 point]*

1.1.3 Construisez l'image sous le nom `hcc`. Fournissez la commande utilisée, l'identifiant de l'image et sa taille. *[0.5 point]*

1.1.4 Lancez d'abord le conteneur sans capacité supplémentaire en transférant le port 8081 de l'hôte vers le port 80 du conteneur et appelez-le `hcc-sans-cap`. Consultez les logs du conteneur `hcc-sans-cap`. Dans un autre terminal, lancez un autre conteneur, nommé `hcc-avec-cap`, à partir de l'image `hcc`, cette fois-ci en ajoutant la capacité `NET_ADMIN` et en transférant le port 8080 de l'hôte vers le port 80 du conteneur. Observez son comportement en analysant ses logs. À partir des logs des deux terminaux, que constatez-vous? Déduisez-en l'utilité de la capacité `NET_ADMIN` pour un conteneur. Supprimez ensuite le conteneur `hcc-sans-cap`. *[1.5 point]*

1.1.5 Avec le conteneur `hcc-avec-cap` en marche, confirmez le fonctionnement du serveur de surveillance depuis votre navigateur. Fournissez une capture d'écran ainsi que l'URL utilisée. *[0.5 point]*

1.2 Validation de l'intégrité et de l'authenticité des images Docker [4 points]

Une application conteneurisée présentée sous forme d'image Docker dans un entrepôt doit être protégée contre des tentatives de modification de la part des utilisateurs/développeurs malveillants. Il est crucial d'avoir des outils qui nous permettent de valider l'intégrité et l'authenticité de l'image avant de la lancer. Pour cela, vous allez utiliser l'outil Skopeo qui supporte le format des images Docker et OCI en général.

Après avoir généré l'image du serveur de vidéosurveillance, vous devez maintenant générer sa signature. Cette signature vous permettra de vous assurer que l'image n'a pas été altérée entre le moment de sa génération et son déploiement. Pour cela, vous utiliserez l'outil Skopeo.

1.2.1 Expliquez la différence entre le digest d'une image, une clé privée de signature, une clé publique de vérification et une signature. Précisez lequel de ces éléments doit demeurer secret. *[1 point]*

1.2.2 Copiez l'image `hcc` dans un répertoire au format OCI avec Skopeo, puis signez son manifeste. Vérifiez ensuite la signature en utilisant uniquement les informations qui pourraient être remises à un administrateur chargé du déploiement. Fournissez les commandes et le résultat de la vérification. *[1 point]*

HCC a été victime d'une cyberattaque interne, un employé mécontent a eu accès aux images Docker. Les investigateurs n'excluent pas la possibilité qu'il les ait modifiées pour insérer une porte dérobée.

1.2.3 Créez l'image `hcc:modifiée` à partir de l'image signée et ajoutez-y le paquet `curl`. Comparez les digests des deux images et expliquez pourquoi ils sont différents. *[1 point]*

1.2.4 Tentez de valider le manifeste de l'image modifiée à l'aide de la signature produite pour l'image originale. Présentez le résultat obtenu et expliquez précisément le lien entre la signature, le manifeste et le digest de l'image. *[1 point]*

1.3 Opérations dans le cluster Minikube [5 points]

Avec l'augmentation du nombre de clients de CSP, la gestion des conteneurs avec Docker devient difficile. L'entreprise décide donc d'utiliser Kubernetes pour orchestrer le déploiement des conteneurs.

Dans cette partie, vous utiliserez `kubectl` pour interagir avec le plan de contrôle de Kubernetes. Vous allez commencer par déployer un objet de type Deployment avec l'image d'un serveur web Nginx et l'exposer. Éliminez les conteneurs déployés avec Docker dans la partie précédente avant d'exécuter les trois instructions suivantes :

```
1 kubectl create deployment serveur-web --image=nginx
2 kubectl expose deployment serveur-web --type=NodePort --port=80
3 minikube service serveur-web
```

1.3.1 De la même manière, créez un objet Deployment nommé `hcc-surveillance` utilisant l'image que vous avez créée du serveur de vidéo surveillance `hcc`. Que remarquez-vous ? *[1 point]*

1.3.2 Identifiez puis expliquez le problème en consultant le fichier log de minikube. *[1 point]*

1.3.3 Déployez l'objet en évitant l'erreur et assurez-vous qu'il est accessible par l'intermédiaire d'un Service nommé `hcc-surveillance`. *[1 point]*

La granularité des objets Kubernetes demandant des ressources commence avec le Pod, l'unité la plus basique implémentée par Kubernetes. Un Pod contient un ou plusieurs conteneurs. Lorsqu'il s'agit de multiplier les instances des conteneurs, Kubernetes multiplie les Pods. Pour assurer un certain niveau de disponibilité, une pratique est de déployer des objets de type ReplicaSet. Par exemple, un ReplicaSet qui déclare trois instances d'un même Pod mène Kubernetes à toujours assurer ce nombre de Pods dans le cluster.

Pour s'assurer de la haute disponibilité du serveur de vidéosurveillance, HCC vous demande d'en créer deux copies supplémentaires. Supprimez le Deployment `hcc-surveillance` tout en conservant son Service avant de créer le ReplicaSet demandé ci-dessous.

1.3.4 Rédigez un manifeste ReplicaSet qui maintient exactement trois Pods du serveur `hcc-surveillance`. Ajoutez aux Pods un label permettant au Service existant de les sélectionner. *[1 point]*

1.3.5 Supprimez simultanément deux des trois Pods, observez le ReplicaSet et expliquez en quoi cette configuration contribue à la haute disponibilité du Service. *[1 point]*

2 Deuxième Partie [13/33]

2.1 TLS en Kubernetes [4.5 points]

2.1.1 Comment pouvons-nous vérifier (avec une commande `kubectl` ou `minikube`) que l'API du serveur écoute en TLS (HTTPS) par défaut ? *[1 point]*

Le client typique pour invoquer l'API de Kubernetes est `kubectl`. Nous allons interagir avec le cluster en utilisant un autre client, tel que `curl`. Sachant que l'API serveur écoute sur HTTPS, il faut que le client ait le bon certificat pour établir la session TLS. Il s'agit d'obtenir un certificat signé par la CA (Autorité de Certification) propre au cluster.

NOTE : par défaut, Kubernetes utilise une CA interne au cluster pour la gestion des certificats des composants internes de Kubernetes. D'autres options sont disponibles également.

2.1.2 À partir de la configuration Kubernetes obtenue avec la commande `kubectl config view`, localisez le certificat de la CA, le certificat client et la clé privée utilisés par le client Minikube. Pour chaque fichier, indiquez son rôle dans l'établissement de la connexion. *[1.5 point]*

Inspectez les données antérieures avec `openssl` :

```
1 openssl x509 -text -noout -in <certificat_client_minikube>
2 openssl x509 -text -noout -in <certificat_ca_cluster>
3 openssl rsa -in <cle_du_client> -check
```

Nous allons réutiliser les certificats et la clé antérieurs avec `curl`, mais une conversion au format `.pem` est peut-être nécessaire. Copiez ces données dans le dossier local et passez à la question suivante.

2.1.3 Convertissez les certificats et la clé antérieurs au format `.pem` en utilisant `openssl`. *[1 point]*

2.1.4 Sans utiliser `kubectl`, interrogez avec `curl` l'endpoint `/version` de l'API Kubernetes en utilisant les certificats et la clé antérieurs. Répétez ensuite la requête sans certificat client et comparez les deux résultats. *[1 point]*

Deuxième séance de TP

2.2 RBAC [8.5 points]

Pour cette partie, vous êtes amené à configurer un système de contrôle d'accès sur le cluster Kubernetes. HCC fait appel à vous et vous informe des détails de votre mission :

Le système de vidéosurveillance des cuves doit être mis à jour. Sophie, membre de l'équipe d'exploitation, est chargée de la mise à jour et du déploiement de la partie applicative. Sophie est appelée à intervenir directement sur le cluster Kubernetes pour mener sa mission. Vous devez vous assurer qu'elle exécute ses tâches de manière isolée, sans interférer avec les ressources du cluster auxquelles elle ne doit pas avoir accès.

Pour répondre aux besoins de HCC, vous allez implémenter une politique de contrôle d'accès RBAC (Role Based Access Control) pour des utilisateurs devant avoir les mêmes permissions dans le cluster actuel. On considère une seule utilisatrice (Sophie), qui se verra attribuer un namespace et travaillera sur son projet dans le cluster. La politique de contrôle d'accès est décrite plus bas.

Vous optez pour une authentification des utilisateurs à l'aide d'un certificat signé par la CA du cluster. Les étapes typiques sont :

1. création d'un utilisateur;
2. obtention des bons credentials (certificats) qui permettent à cet utilisateur d'être authentifié à chaque accès au cluster;
3. création des objets Kubernetes nécessaires pour ce nouvel utilisateur : namespace, Role, RoleBinding, etc., en fonction de la politique visée.

Commencez par créer un namespace **operations** pour la nouvelle utilisatrice :

```
1 kubectl create ns operations
```

2.2.1 Quelle est votre démarche pour :

- (a) créer une nouvelle utilisatrice appelée **sophie** sur Linux. *[0.5 point]*
- (b) obtenir un certificat signé par l'autorité de certification propre au cluster avec **openssl** ? *[1.5 point]*

Nous allons configurer les credentials de la nouvelle utilisatrice dans le cluster avec la commande :

```
1 kubectl config set-credentials sophie --client-certificate=<chemin_vers_sophie.crt> --  
  client-key=<chemin_vers_sophie.key>
```

Le résultat doit être : **User "sophie" set.**

Ensuite, nous allons configurer le contexte pour l'utilisatrice Sophie avec la commande :

```
1 kubectl config set-context sophie-context --cluster=minikube --namespace=operations --  
  user=sophie
```

Résultat : **context "sophie-context" created**

2.2.2 Quelle est votre interprétation du résultat des commandes suivantes ? *[1 point]*

```

1 kubectl --context=sophie-context get pods
2 kubectl --context=sophie-context get -n kube-system pods
3 kubectl -n operations --context=sophie-context get pod

```

La politique de contrôle d'accès RBAC que nous ciblons permettra à l'utilisatrice **sophie** de créer, lire et modifier les objets Kubernetes Deployment, ReplicaSet, Pod et Service dans son namespace **operations**. Les permissions en langage Kubernetes sont des “verbs” et nous utiliserons les verbes suivants : **list**, **get**, **watch**, **create**, **update**, **patch** et **delete**.

La politique implique l'attribution d'un rôle auquel des permissions sont affectées. Un utilisateur est affecté à un rôle et obtient ainsi les permissions de ce rôle. Concrètement, il s'agit de créer un objet Role et un objet RoleBinding. Nous allons décrire ces objets dans des fichiers YAML. Le rôle initial autorise la consultation des Services, mais pas encore leur création.

Dans un fichier `role-devops.yaml`, ajoutez :

```

1 kind: Role
2 apiVersion: rbac.authorization.k8s.io/v1
3 metadata:
4   namespace: operations
5   name: devops
6 rules:
7 - apiGroups: ["apps"]
8   resources: ["deployments", "replicasets"]
9   verbs: ["list", "get", "watch", "create", "update", "patch", "delete"]
10 - apiGroups: [""]
11   resources: ["pods"]
12   verbs: ["list", "get", "watch", "create", "update", "patch", "delete"]
13 - apiGroups: [""]
14   resources: ["services"]
15   verbs: ["list", "get", "watch"]

```

Dans un fichier `rolebind-devops.yaml`, ajoutez :

```

1 kind: RoleBinding
2 apiVersion: rbac.authorization.k8s.io/v1
3 metadata:
4   name: devops-role-binding
5   namespace: operations
6 subjects:
7 - kind: User
8   name: sophie
9   apiGroup: rbac.authorization.k8s.io
10 roleRef:
11   kind: Role
12   name: devops
13   apiGroup: rbac.authorization.k8s.io

```

Créez par la suite les objets Kubernetes afférents :

```

1 kubectl create -f role-devops.yaml
2 kubectl create -f rolebind-devops.yaml

```

Testez que le résultat obtenu est conforme à la politique RBAC. Par exemple :


```
1 kubectl -n operations --context=sophie-context get pod
```

Cette commande ne devrait plus indiquer une erreur d'autorisation.

2.2.3 Continuez en créant un Deployment nommé `hcc-video-ops` avec l'image `hcc` du serveur de vidéosurveillance en tant qu'utilisatrice `sophie` dans l'espace `operations`. *[1 point]*

Le Service lié au déploiement du serveur de vidéosurveillance doit maintenant être exposé. Référez-vous aux opérations sur Minikube de la Section 1.3. Tentez la commande suivante :

```
1 kubectl --context=sophie-context expose deployment hcc-video-ops --type=NodePort --port=80
```

2.2.4 Comment interprétez-vous le résultat de l'opération précédente ? *[1.5 point]*

2.2.5 Comment pouvez-vous modifier la politique RBAC pour résoudre cette situation ? *[1.5 point]*

2.2.6 Est-ce que le Service est devenu accessible ? Si oui, listez les commandes et fournissez une capture d'écran de votre fureteur accédant au Service, ou montrez le résultat obtenu avec `curl`. *[1.5 point]*

3 Troisième Partie [7/33]

Pour renforcer la sécurité de sa nouvelle infrastructure, HCC fait appel à vos services pour effectuer un test de pénétration. Vous convenez du scénario suivant : vous êtes un concurrent de HCC et également client de CSP. Pour vous permettre d'effectuer vos tâches d'administration, CSP vous crée un conteneur administrateur.

3.1 Lancez un conteneur Ubuntu nommé `maintenance-csp` avec l'option `--privileged`. Comparez ses capacités avec celles d'un conteneur Ubuntu ordinaire. *[0.5 point]*

3.2 Depuis le conteneur privilégié, accédez au système de fichiers de la machine hôte et créez le fichier témoin `/tmp/hcc-compromis.txt`. Expliquez les privilèges ayant rendu cette opération possible. *[1.5 points]*

3.3 Assurez-vous d'avoir un accès permanent et furtif à l'hôte. *[1 point]*

3.4 Modifiez l'image du serveur de vidéosurveillance de HCC sur Kubernetes en y ajoutant un programme malveillant (exemple : `nmap`). *[1 point]*

3.5 Quelles sont les vulnérabilités que vous avez identifiées ? Citez-en au moins trois. *[1 point]*

3.6 Quelles contre-mesures recommandez-vous pour empêcher cette attaque ? *[1 point]*

3.7 Implémentez une de ces contre-mesures. *[1 point]*